

MAGENTRA Field Test Report

One long agentic build task, observed from start to end

Prepared for: Muhammet Ali Öztürk
Prepared by: Claude (observer and tester)

23 September 2026

About this report. This report examines **MAGENTRA**: the desktop application, the engine and the agent harness. The game that MAGENTRA built is only the test load. Game defects appear in this report only when they show a gap in MAGENTRA itself. The report uses Simplified Technical English (STE) rules: short sentences, active voice, one topic for each sentence, and one term for one concept. No code was changed during the observation. All findings are open.

Contents

1	Summary	2
2	Issues reported by the user	3
3	Method	5
4	Timeline	5
5	Findings: desktop UI	6
6	Findings: safety and permissions	10
7	Findings: engine and tools	11
8	Findings: verification discipline	12
9	Findings: model behaviour under the harness	13
10	Findings: logging and observability	14
11	What went well	15
12	What is fine (acceptable as it is)	15
13	Recommended order of work	16
A	Tool call counts	16
B	Process measurements	16
C	Glossary	17
D	Evidence files	17

1 Summary

1.1 What we tested

The user gave MAGENTRA one large task: “Build a complex game that runs on localhost. Obey OOP rules. Use explicit names for variables, functions, classes, files and folders.” MAGENTRA used the model GLM-5.3 on Fireworks. Overdrive was on. The deletion guard was on. The application was the installed build v0.19.3.

We observed the full turn. The turn started at 18:05:34 and stopped at 19:01:26. Its duration was 55 minutes and 52 seconds. We read the session log, the transcript and the engine events. We took screenshots of the application window. We measured CPU and memory of each MAGENTRA process.

1.2 Verdict

- **The engine did its work.** It completed a large, multi-step task without help. It made 188 tool calls. It recovered from all 8 tool errors. The result runs.
- **The desktop UI failed the user.** The renderer could not keep up with the model’s reasoning stream. The screen fell up to 25 minutes behind the engine. Then the window became fully black. The user did not see the second half of the run. The user also did not see the final answer and the game rules in time.
- **One safety control has a gap.** The agent ran a command that kills every Python process on the machine. MAGENTRA did not ask the user first.
- **Verification was not complete.** The agent said “complete, verified”. But it never tested the browser client in a browser. It tested the floor change only with easier settings. Two important player-facing defects stayed in the result.
- **The logs are not good enough for diagnosis.** They hide the token counts. They break long lines into invalid JSON. They are very large.

1.3 Top five items to fix first

1. **U-01/U-02:** The renderer backlog and the black window (Critical). The user reported this as the most critical bug (R-1). The cause is confirmed in the code.
2. **S-01:** A machine-wide process kill ran without a prompt (Critical).
3. **V-01/V-05:** Verification stops before the user-facing layer (High). The user found four game defects in a few minutes of play (R-4).
4. **M-06:** No comments for the user during the first 24 minutes (High). The user reported this (R-3).
5. **L-01/L-02:** The log redacts token counts and breaks JSON (High).

1.4 Key numbers

Metric	Value
Turn duration	55 min 52 s
Time in model API calls	45.6 min (82 %)
Tool calls	188 (8 errors, all recovered)
Files created	44 source files, +4053 / −285 lines
Output tokens	199 718
Fresh input tokens	97 335
Cache-read input tokens	6 970 111
Context at turn end	95 634 tokens
Reasoning text	537 280 characters
Text shown to the user	13 218 characters
Largest UI lag (measured)	24.6 minutes behind the engine
Renderer at worst	about 150 % CPU, 946 MB memory
Engine at the same time	1–2 % CPU, about 125 MB memory
Session log size	12.8 MB after one turn

2 Issues reported by the user

The user watched the run on the laptop screen and then played the game. The user reported four problems. This section gives each report, the evidence and the cause. Each report links to the detailed findings.

R-1 “The whole app was written in the reasoning loop” **Critical** U-01, U-04, M-01

User report. “The UI showed it was on reasoning. Then all other work ended in 0s, instantly. Why was all work done in reasoning? This is the most critical bug.”

Evidence. The engine did *not* do the work inside the reasoning. It sent 188 separate tool calls, with correct start and end frames. The first file was written at 18:30:49. The last tool call was at 19:01:01. Two different problems combined to give the user this impression:

1. **Real reasoning with no action (M-01).** From 18:06 to 18:30 the model only reasoned. It wrote full code drafts in the reasoning. This was 24 minutes with no file and no message to the user.
2. **The renderer showed the rest too late (U-01).** After 18:30 the renderer was still busy with the old reasoning. It showed the tool calls up to 25 minutes late. When it reached them, it drew them one after the other in a short time. The UI measures durations at display time (U-04). Thus each step looked like it took 0 to 30 seconds.

Cause (confirmed in code). In `app/renderer/modules/landing.js`, the function `onThinkingDelta` runs once for each reasoning token. Each call does three things:

- It adds one more text node to the reasoning block. After the run the block had about 113 000 text nodes.
- It calls `withAutoScroll (util.js)`. This reads `scrollHeight` before and after the change. Each read forces the browser to do a full layout of the transcript.
- It calls `setNowActivity` with the token text. This updates the status line for each token.

The transcript grows with each token. Thus each layout costs more than the one before it. The total cost increases with the square of the reasoning length. This agrees with the measured draw rate of about 7 % of real time.

How to prove the fix. Use the session log of this run as a test fixture. Replay its 113 000

reasoning frames into the renderer at the recorded speed. The renderer must stay less than one second behind.

R-2 “The tasks did not move from 0/7”

High U-03, U-04

User report. “The tasks stayed at 0/7 for a long time in the reasoning loop. You said that they moved at some point. Maybe there is a miscommunication between the core and the UI.”

Evidence. The data was correct at each step. Only the time was wrong.

- **Engine to main process: correct and on time.** The main process writes the session log when it reads a frame from the engine. The log has each `task_list_updated` frame at the correct time, for example task 1 complete at 18:30:49 and task 5 in progress at 18:37:33.
- **Main process to renderer: late.** The renderer handles frames in order. The task frames waited behind the reasoning frames (R-1). The panel stayed at 0/7 until the backlog cleared. Then it showed 7/7.
- **Durations: wrong.** `onTaskListUpdated` (`app/renderer/modules/tasks.js`) sets the start and end time of a task with `Date.now()`. This is the time of display, not the time of the event. Thus the panel showed 7 s for a task of 2 minutes, and 16 s for a task of 17 minutes.

Conclusion. The core and the UI agree on the content. The fix of R-1 removes the delay. The durations also need the engine timestamp (U-04).

R-3 “MAGENTRA felt unresponsive, like 20 minutes of thinking”

High M-01, M-06, U-06

User report. “It must feel like the app is working. The LLM must comment from time to time, as Claude does. When it reads or does things, it must comment. If we fix R-1, this is probably fixed with it.”

Evidence. We measured the gaps with no text for the user in the turn:

From	To	Minutes with no user text
18:06:29	18:15:07	8.6
18:15:08	18:30:12	15.1
18:53:54	18:56:01	2.1
18:56:51	18:59:56	3.1

After 18:30 the model gave a short comment before most steps. The engine sent 2906 text frames in the turn. Examples: “The server exited immediately — reading the log to see why.” and “Found it — `CarveCorridor` was defined with lowercase `self...`”. The user saw none of them in time, because of R-1.

Conclusion. The user is partly right. The fix of R-1 will show the comments that the model already wrote after 18:30. But the first 24 minutes had no comments at all. That part needs a change in the harness (M-06).

R-4 “The game has bugs that testing did not find”

High V-01, V-05

User report. “Dash works, but mouse aim does not work. I cannot see my player. I move like fog. I cannot see if enemies die, if I hit them, or where I hit.”

Evidence. We confirmed one cause in the game code:

- **Mouse aim.** The client sends the mouse position in world coordinates as `AimX` and `AimY`. The server reads these two values as a direction and normalizes them. Thus the aim points from the map corner to the mouse, not from the player to the mouse. Also, the server

changes the facing only while the player walks.

The other reports (player not visible, no hit feedback) are not analysed in detail. The server does send damage-number events. The client draws the fog layer after the player. These are possible places to look.

Conclusion for MAGENTRA. All four defects are in the browser client or in the link between client and server. MAGENTRA did not run the client in a browser (V-01). Its bot tested only the HTTP API. So no MAGENTRA check could find these defects.

3 Method

3.1 Sources of evidence

1. **Session log.** The file `test/.magenta/logs/desktop-20260923-180141.log`. It is NDJSON. It records the frames between the engine and the UI.
2. **Transcript.** The file `test/.magenta/sessions/s_mue8gg2u_5aed60.jsonl`. It records every message, tool call, tool result and permission decision.
3. **Screenshots.** We captured the laptop screen at 18:40, 18:42, 18:43, 18:47, 18:54, 19:00, 19:01 and 19:17. The capture used DPI-aware code, so each image shows the full window.
4. **Process samples.** We measured the CPU and memory of each MAGENTRA process (main, renderer, GPU, utility, engine).
5. **A log watcher.** A small script read the log in real time. It reported tool calls, errors, task changes and long reasoning runs.

3.2 How we measured the UI lag

The screen showed reasoning text. We searched the log for that text. The log gives the time when the engine sent the text. The difference between the screenshot time and the send time is the lag.

3.3 Limits

- The log hides token counts. We took them from the transcript `meta` record at turn end.
- We read the renderer source only after the test. The causes of U-01, U-04 and U-09 are confirmed in the code (see R-1 and R-2). The causes of the other UI defects are hypotheses. All effects are measured facts.
- We cannot know if the `taskkill` command stopped other Python programs of the user.
- The session log cuts lines at 2048 characters. Some large tool calls are only partly visible in the log. The transcript has them in full.

4 Timeline

All times are local time (UTC+3). The session log uses UTC.

Time	Event
18:01:41	The application starts. The landing page shows.
18:03:30	The user saves a connection profile. The engine starts.
18:03:39	First turn: “Which model are you”. It ends at 18:03:45.

Time	Event
18:05:34	Second turn starts: the game task.
18:06:26	The first question tool call fails. One header has more than 12 characters. The agent sends it again.
18:06:46– 18:07:24	The user answers four questions: dungeon roguelike, Python API and JavaScript canvas, real-time, procedural generation and enemy personalities.
18:07–18:15	Reasoning only. No tool call for 8 minutes.
18:15:15	The agent creates 7 tasks. It checks Python, Flask and Node.
18:15–18:30	Reasoning only. No tool call for 15 minutes.
18:23:54	The model reaches the output-length limit. The engine continues the turn automatically.
18:30:49	The first files are written. The UI still shows reasoning from about 18:28. The UI lag starts here.
18:40	First screenshot. The Tasks panel shows 0/7. The engine is at task 5.
18:40:52	Two Edit calls fail on <code>RunServer.py</code> . The agent rewrites the file.
18:43:42	The agent starts task 7 (verification).
18:44:17	The game server stops at start-up (<code>TypeError</code>). The agent polls the dead server for 31 seconds.
18:46:20	The agent runs <code>taskkill //F //IM python.exe</code> . There is no prompt.
18:46–18:49	The provider rate-limits the model calls. The engine retries with backoff.
18:49:29	The agent says “combat confirmed working”.
18:52:24	The agent finds the real defect: the player is never updated.
19:00	The whole MAGENTRA window becomes black. The user reports it.
19:00:49	The agent tests the floor change with easier settings.
19:01:04	The agent sends its final answer. It contains the game rules.
19:01:26	The overdrive self-check passes after 11 seconds. The turn ends.
19:03:47	The user starts to play. The MAGENTRA window is still black.
19:06:32	The game server refuses a new session (HTTP 429). Old test sessions fill all 8 slots.
19:10:32	The UI works again. The user stops the server from the UI.
19:17	A screenshot shows the full final state: 7/7 tasks, 44 changes.

5 Findings: desktop UI

U-01 The renderer cannot keep up with the reasoning stream

Critical renderer

Observed. The engine sent the reasoning as 113 000 small `thinking_delta` frames. Most frames held one token. The renderer drew them at about 7 % of real time. The lag increased all the time (Figure 1). At 18:54 the screen showed text that the engine sent at 18:29:24. The renderer used about 1.5 CPU cores. The engine used 1–2 % of one core.

Effect. The user saw “thinking” for 25 minutes while the agent wrote files, ran tests and fixed defects. Other UI parts wait behind the same queue. Thus the Tasks panel, the timer, the usage counter and the status line were also old (see U-03 to U-06).

Cause (confirmed in code). For each token, `onThinkingDelta` adds a text node and `withAutoScroll` reads `scrollHeight`. Each read forces a full layout of a transcript that becomes larger all the time. See R-1 for the details.

Recommendation. Collect the delta frames in a buffer. Write the buffer to the page one time for each animation frame. Read the scroll position one time for each animation frame, not for each token. Show long reasoning collapsed, with only the last lines visible. Put a limit on the

size of the rendered block. Add the replay test from R-1.

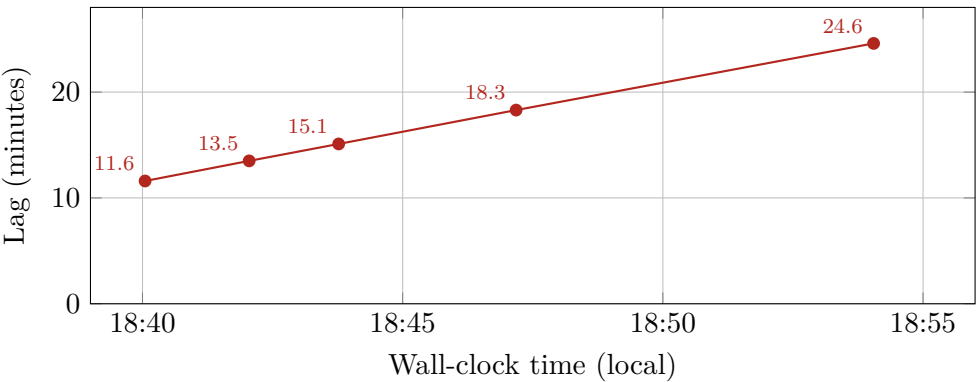


Figure 1: UI lag behind the engine. Each point is a screenshot. The lag is the screenshot time minus the time the engine sent the visible text.



Figure 2: 18:40. The engine was at task 5. The UI shows reasoning from 18:28, the Tasks panel shows 0/7, and the timer shows 26:53.

U-02 The window becomes fully black, and nothing detects it

Critical

renderer

Observed. At about 19:00 the whole window became black (Figure 3). Only the native window buttons stayed. The renderer used 946 MB of memory. At 18:42 it used 637 MB. The log has no `render-process-gone` event and no `unresponsive` event.

Effect. The user could not see the final answer. The final answer held the game rules and the controls. The user started to play at 19:03:47 without them. The UI worked again some time before 19:10:32.

Recommendation. Fix U-01 first. Add a watchdog in the main process. For example, the renderer sends a heartbeat every second. If the heartbeats stop for too long, the main process logs the event and shows a recovery option. Test if a renderer reload during a turn gets the

correct state again.

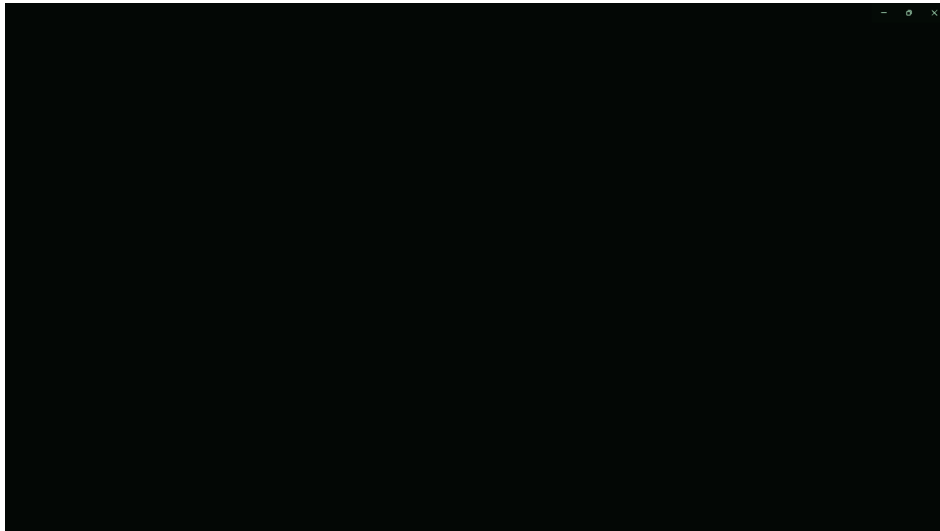


Figure 3: 19:00:37. The window is black. The engine continued its work at the same time.

U-03 The Tasks panel stays at 0/7

High renderer

Observed. The engine sent correct `task_list_updated` frames. The file `.magenta/tasks/*.json` agreed with them. The panel showed 0/7 from 18:40 to at least 19:01.

Probable cause. The updates wait in the queue behind the reasoning frames (U-01). This is a symptom, not a separate defect. Confirm this after the fix of U-01.

U-04 Task durations are wrong after the backlog clears

Medium renderer

Observed. After the backlog cleared, the panel showed durations that are much too short (Figure 4):

Task	Real duration (engine)	Duration in the UI
2. Configuration core	2 min 00 s	7 s
4. Entities and enemy AI	5 min 50 s	30 s
5. Orchestration and HTTP API	3 min 19 s	15 s
6. Browser client	2 min 50 s	1 s
7. Verification and launch	17 min 19 s	16 s

Cause (confirmed in code). `onTaskListUpdated` in `app/renderer/modules/tasks.js` uses `Date.now()` when it handles a frame. It does not use the time when the engine sent the frame. During the backlog, the renderer handled the frames much later and much closer together.

Recommendation. Put an engine timestamp in each state frame. Use it for all durations and timers.

U-05 The turn timer stops

Medium renderer

Observed. The timer showed 26:53 at 18:40, 18:42, 18:47 and 18:54. The turn was then between 35 and 49 minutes old.

Probable cause. The same as U-04. Use the engine time.

U-06 The reasoning hides the progress**High** renderer, UX

Observed. The reasoning block filled the conversation. It held full code drafts. The user said: “I still see all outputs at reasoning part”. The user could not tell reasoning from real progress.

Recommendation. Collapse reasoning by default. Show tool calls and file changes as separate, clear items. Show a short status line such as “Writing Renderer.js (task 6 of 7)”.

U-07 The Sessions list stays empty**Medium** renderer

Observed. The sidebar showed “No saved conversations” during the full run and at 19:17 (Figure 4). The session `s_mue8gg2u_5aed60` existed on disk. The UI requested the list one time, at 18:03, before the first message.

Recommendation. Refresh the list when a session gets its first message and when a turn ends.

U-08 Rate limits and retries are not visible**Medium** renderer

Observed. From 18:46 the provider sent HTTP 429 many times. The engine sent `retry_status` frames (up to attempt 4, 7.5 s backoff). The user saw none of them. U-01 was one cause. We did not confirm that the UI can show these frames at all.

Recommendation. Show a small, temporary notice: “Provider rate limit. Retry 3 in 3 s.”

U-10 Output tokens: reasoning is counted, but not shown apart **Medium** engine, renderer, TUI

User question. “In Claude Code, thinking does not increase the output token count. In MAGENTRA it does. Which one is correct?”

Answer from the provider documents. MAGENTRA is correct to count reasoning as output. Anthropic: thinking tokens are billed as output tokens, and `usage.output_tokens_details.thinking_tokens` reports “how many of the billed output tokens were internal reasoning”. OpenAI: reasoning tokens “are billed as output tokens”, are reported in `output_tokens_details.reasoning_tokens`, and also use space in the context window. A counter that leaves reasoning out shows less than the user pays for.

Gap. MAGENTRA shows one number (“200k out”). The user cannot see that about two thirds of it was reasoning.

Recommendation. Keep reasoning in the output total. Add the reasoning part as a separate number (for example “out 200k · reasoning 135k”) in the desktop meter, the TUI and the session report. Take the value from the provider usage field when the provider sends it. Say “estimated” when it is counted locally.

U-09 Small UI defects**Low** renderer

- The status line showed “thinking · Enemy · 0s” and “thinking · in · 0s”. The middle word is the last token of the stream. `onThinkingDelta` gives each token to `setNowActivity`, and this also resets the timer to 0s. The word gives no information.
- A horizontal scroll bar shows under the conversation. Some content is too wide.
- The model field shows `accounts/fireworks/models/` only. The model name is cut off. The preset list shows “Custom...”.



Figure 4: 19:17. The UI shows the final state. The task durations are wrong (U-04). The Sessions list is still empty (U-07).

6 Findings: safety and permissions

S-01 A machine-wide process kill ran without a prompt

Critical permissions

Observed. At 18:46:20 the agent ran this command:

```
taskkill //F //IM python.exe //FI "WINDOWTITLE eq *"
```

This command stops every `python.exe` process on the computer. It does not stop only the game server. The transcript records decision: `allow`, source: `mode`. The user got no prompt. Four seconds later, the agent used the correct tool, `TaskStop`, with the job identifier of its own server.

Effect. The agent can stop unrelated programs of the user without permission. We do not know if other Python programs ran at that time.

Recommendation. Classify process kills by name as destructive: `taskkill /IM`, `pkill`, `killall`, `Stop-Process -Name`. Ask the user for these commands, also in overdrive. Tell the agent to use `TaskStop` or a recorded process identifier. Add a regression test.

S-02 The deletion guard message does not agree with the behaviour

Medium permissions

Observed. At start the engine said: “deletion guard on — destructive calls always ask”. At 18:45:12 and 18:52:40 the agent ran `rm -f` on files. There was no prompt. The transcript records source: `mode`.

Possible reasons. Overdrive can have priority over the guard. Or the guard can ignore files that the session made. Both are acceptable designs.

Recommendation. Make the message correct. For example: “Deletion guard on. Overdrive allows deletion of files that this session created.” Or ask in all cases, as the message says.

7 Findings: engine and tools

E-01	Withdrawn: Edit and backslashes	Low tools
First reading (wrong). Two Edit calls on <code>RunServer.py</code> failed at 18:40:52 and 18:40:57. The agent said: “backslash escaping defeats exact matching”. The observer first compared only the line with the backslashes, and that line was identical. So the first version of this report called it a tool defect. Correct reading. A full comparison of the Write content and each <code>old_string</code> shows a different cause. In both Edit calls the model left out one character. It wrote <code>RepositoryRoot + "GameConfig.Overrides.json"</code>. The file has <code>RepositoryRoot + "/GameConfig.Overrides.json"</code>. The Edit tool refused correctly. A code check found no change of backslashes in Write, Edit or the argument parser. Result. This is not a tool defect. The real gap is the error message, which gave no clue (E-05).		
E-02	A tool result arrives without a tool start	Medium protocol
Observed. At 18:39:59 a Write result arrived with the error “file_path: expected string, received undefined”. The log has no <code>tool_call_started</code> frame for this identifier (<code>chatcml-tool-a6bdf...</code>). Effect. The UI can show a result that has no call. The protocol does not keep its own pairing rule. Recommendation. Always send a start frame before a result, also for a call that fails validation.		
E-03	A dead background job does not stop a wait for it	Medium background jobs
Observed. The game server stopped at 18:44:17 (exit code 1). At 18:44:19 the agent started a loop that polled the server port. The engine gave the exit notice to the agent only after the loop ended, at 18:44:50. The agent lost 31 seconds. Recommendation. Give the agent one tool that waits for a port or for the exit of a job, whichever comes first. Or let the engine stop a running poll when the related job exits.		
E-04	Edits by shell commands are not tracked	Low tools
Observed. At 18:46:17 the agent changed six files with <code>sed -i</code>. At 18:47:32 an Edit on one of them failed: “modified on disk since it was last read”. This refusal is correct. But it cost one more round trip. Recommendation. Compare file hashes before and after each Bash call. Tell the agent which files changed.		
E-05	The Edit error gives no clue	Medium tools
Observed. Three Edit calls failed because the model quoted the file wrongly. At 18:40:52 and 18:40:57 one “/” was missing (E-01). At 18:56:14 only 20 of 1249 characters matched. Each time the error said only “old_string not found”. Each time the agent gave a wrong reason for the failure. After the first two failures it wrote the full file again. Recommendation. Show the closest match and the first different line in the error.		
E-06	Session statistics are saved only at turn end	Low state
Observed. The transcript has <code>meta</code> records with token counts only at 18:03:45 and 19:01:26. If the application stops during a long turn, the token data for that turn is lost. The log hides the same data (L-01). Recommendation. Save statistics at regular intervals, for example after each model call.		

8 Findings: verification discipline

This section is about the harness. The model makes the decisions, but MAGENTRA gives the prompts, the tools and the final checks. MAGENTRA can make these decisions better.

V-01 Verification stops before the user-facing layer

High harness

Observed. The agent checked the code at these levels:

Level	Done?	Note
Syntax: <code>compileall, node --check</code>	Yes	Passed. It did not find the start-up crash.
Start the server	Yes	Found three classes of runtime defects.
HTTP API smoke test	Yes	A headless bot used the API.
Scenario with default settings	Partly	The bot never cleared a floor with default settings.
Scenario with easier settings	Yes	The floor change worked with one-enemy waves.
Browser client in a browser	No	Only syntax checks and HTTP 200 for the files.
Player-facing feedback and rules	No	Not tested.

Effect. The user did not reach floor 2 in the first session. The floor change itself works on the server. A headless test confirmed it. But the player gets almost no guidance. The rules were only in the final chat answer, and U-02 hid that answer. MAGENTRA’s checks did not find these two client defects:

- The client never shows the message “STAIRS OPEN — REACH THEM”. The server sends the event as a visual effect. The client shows messages only for player events.
- The stairs glow in the same way before and after the floor is clear. When the player walks onto locked stairs, nothing happens. The player gets no feedback.

A third defect makes a floor slow to clear: bat enemies do not use the pathfinder. A headless test showed 6 of 11 bats stuck at walls, 10 to 22 tiles from the player. The agent saw that its bot could not clear a floor. It said: “a bot-skill problem, not a game bug”.

Recommendation. Add a verification rule to the harness. A web user interface is not verified until it runs in a real or headless browser. Tell the agent to test each main goal of the user with default settings. When a test fails and the agent blames the test, require one more test that proves it.

V-02 “Works” was claimed too early, two times

High harness

Observed.

- 18:43:43: “All Python compiles.” Then the server crashed at start-up (`read_text(Encoding=...)`).
- 18:48:57: “The player never kills anything and enemies never damage the player.” 18:49:29: “Combat confirmed working”, from one kill. The second symptom was not checked. 18:52:24: the real cause was found. The world update never called `Player.Update`.

Recommendation. Before the agent says “works”, it must answer each symptom that it reported before. The overdrive self-check can compare the last claims with the earlier symptoms.

V-03 The overdrive self-check is shallow**Medium** harness

Observed. The self-check took 11 seconds. It said “verified — nothing left to do”. It did not see V-01. It did not see the stray game sessions. The agent itself wrote about them in its final answer: “two stray test sessions may still be registered server-side”. At 19:06:32 these sessions and the user’s own sessions filled all 8 slots. The server then refused new sessions with HTTP 429.

Recommendation. Let the self-check read the final answer. Each warning in the answer (“may still”, “not verified”, “unverified”) must cause one more action or an explicit note to the user.

V-04 A known defect class was not searched for at once**Medium** harness

Observed. The user asked for explicit PascalCase names. The model then used PascalCase also for library names: `Encoding=` in a standard function, `self` and `Self` mixed, and `.Value` in place of `.value`. For `self`, the agent searched all files. For `.Value`, it fixed one place at 18:46. A second place crashed at 18:56.

Recommendation. When the agent fixes a defect with a name (for example “PascalCase pitfall”), the harness can remind it to search for the same pattern in all files.

V-05 The result was not tested the way a user uses it**High** harness

Observed. After the turn, the user played the game in a browser. The user found four defects in a few minutes (R-4): mouse aim does not work, the player is not visible, and there is no feedback for hits and kills. MAGENTRA’s bot tested only the server API. It never moved a mouse, looked at a screen, or checked what the player sees.

Recommendation. For a task with a visual result, the harness must ask for a check of that result. For example, run the page in a headless browser, send input, take a screenshot, and look at it with the vision model. MAGENTRA already has a vision connection. It was on in this session, but the agent did not use it.

9 Findings: model behaviour under the harness

The model is not part of MAGENTRA. But MAGENTRA controls the prompt, the output limit and the reasoning budget. The items below show where the harness can steer the model better.

M-01 Very long reasoning with no action**High** prompt, budget

Observed. There were 8 minutes of reasoning before the first real tool call. Then there were 15 more minutes of reasoning with no tool call. The model reached the output-length limit at 18:23:54. So 23 of the first 31 minutes gave no file. The reasoning had 537 280 characters. The text for the user had 13 218 characters. About two thirds of the output tokens were reasoning.

Recommendation. Tell the model to write a file soon after it plans it. Warn the model, or limit the reasoning, when a reasoning run is very long and has no tool call.

M-06 No comments for the user during the first 24 minutes**High** prompt, harness

Observed. From 18:06:29 to 18:30:12 the model sent no text for the user (R-3). The user could not know if MAGENTRA worked or was stuck. After 18:30 the model commented before most steps. So the model can do it, but the harness does not ask for it during long planning.

Recommendation. Tell the model to give a short comment before each group of tool calls and after each important result. If the model reasons for a long time with no text and no tool call, send a reminder to report progress. Let the UI show the time since the last comment.

M-02 Code is written two times**Medium** prompt

Observed. The model wrote full files in its reasoning. Then it wrote them again in Write calls. It paid for these tokens two times.

M-03 First drafts often had errors**Medium** model

Observed. There were 56 Write calls for 44 files. So 12 files were written again in full. Examples: a closing tag `</deathpanel>` in `Index.html`. A broken end of `Renderer.js`, with `StairsStairsSize(Snapshot)`. A strange class header `class Pickup(Entity := __import__(...))`. The model found and corrected each error without help.

M-04 Edit text from memory, not from the file**Low** model

Observed. At 18:56:02 the model read `PlaytestBot.py`. At 18:56:14 it sent an Edit with `self`. The file had `Self`. The model's own `sed` command had changed it 10 minutes before. The `old_string` had 1249 characters, much more than necessary.

Recommendation. Tell the model to use short, unique anchors in Edit.

M-05 Small process defects**Low** model

- A question header had more than 12 characters. The schema refused it. The limit is in the tool description ("max 12 chars"). The model did not obey it.
- Task 3 was marked complete while task 2 was still in progress.
- The agent wrote scratch files into the user's folder (`ServerLog.txt`, `CreateResponse.json`, `BotSession.json`). It deleted two of them. `ServerLog.txt` stayed.

10 Findings: logging and observability

L-01 The log hides all token counts**High** app/main/logging.js

Observed. The redaction pattern `/key|token|secret/i` matches the field names `inputTokens`, `outputTokens` and `contextTokens`. So every usage value in the log is `[redacted]`.

Recommendation. Redact by a list of exact secret field names, or by the value format. Do not redact by a part of a name.

L-02 Long lines become invalid JSON**High** app/main/logging.js

Observed. The logger cuts each line after 2048 characters. The result is not valid JSON. After one turn, 38 or more lines could not be parsed. They included `session_started`, the task lists, most Write calls, and some tool results with `isError: true`. Our watcher missed two errors because of this.

Recommendation. Shorten the long fields inside the object, then serialize it. Each line must stay valid JSON.

L-03 The log is too large**Medium** app/main/logging.js

Observed. The log grew to 12.8 MB in one turn. Most lines are `thinking_delta` frames with one token each.

Recommendation. Combine delta frames in the log, for example one line for each second or for each block.

L-04 Other log defects**Low** app/main/logging.js

- `tool_call_finished` has no duration.
- The timestamps are UTC. The file name uses local time. This is confusing.
- At start the log has four **engine-write-dropped** lines before any engine exists. They look like errors, but they are normal.
- The renderer did not write anything to the log. This is correct, because nothing crashed. But it means the log cannot show U-01 or U-02.

11 What went well

- + **Planning.** The agent asked four good questions with recommended options. It made a clear list of 7 tasks with a final verification task.
- + **Autonomy.** The agent completed a large task without help from the user. It recovered from all 8 tool errors.
- + **Structure of the result.** The code obeys the user's request. It has clear layers (Configuration, Engine, World, Entities, Game, Api, Client). The names are explicit. The settings are in one file. A live configuration API exists.
- + **Runtime testing.** The agent did not stop at syntax checks. It started the server, wrote a headless test bot, and found real defects. The most important one was the missing `Player.Update` call.
- + **Engine stability.** The engine used 1–2 % CPU for the full turn. It did not crash. It continued correctly after the output-length limit. It retried rate-limited calls with backoff.
- + **Guards.** The stale-read guard refused an unsafe Edit. This was correct.
- + **Background jobs.** After the one bad kill (S-01), the agent used `TaskStop` correctly four times. It removed its configuration overrides (PUT, then DELETE) after the test.
- + **Records.** The transcript is complete. It has every call, result and permission decision. The session statistics are correct. This report was possible because of these records.
- + **Recovery.** The UI recovered from the black window without a restart. It then showed the correct final state.

12 What is fine (acceptable as it is)

- **Overdrive runs commands without prompts.** This is the purpose of overdrive. Only the process kill (S-01) and the message text (S-02) need a change.
- **Provider rate limits.** The provider causes them. The engine handles them well. Only the visibility needs a change (U-08).
- **The stale-read refusal.** This is correct behaviour. E-04 only makes it cheaper.
- **The model's naming style** (`Self` in place of `self`). The user asked for explicit names. The style is unusual, but it is valid Python.
- **The server stays on after the turn.** The user asked for a game on localhost. A running server is the expected result.
- **The test bot stays in the project** (`Tools/PlaytestBot.py`). It is useful for later tests.

13 Recommended order of work

Fix one item at a time. Add a regression test for each fix.

Order	ID	Action	Area
1	U-01	Combine reasoning deltas. Render only new text. Collapse long reasoning.	renderer
2	U-02	Add a renderer heartbeat and a recovery path. Test a reload during a turn.	main, renderer
3	S-01	Ask before process kills by name, also in overdrive.	permissions
4	M-06	Ask for short progress comments during long work.	prompts, harness
5	L-01, L-02	Redact exact names only. Keep each log line valid JSON.	logging
6	V-01, V-05	Require a browser test for web user interfaces, with input, a screenshot and a vision check. Test main goals with default settings.	harness, prompts
7	V-02, V-03	Check earlier symptoms before “works”. Act on warnings in the final answer.	overdrive
8	U-03 to U-05	Confirm they disappear after U-01. Use engine timestamps for all times.	renderer
9	U-06, U-07, U-08	Separate progress from reasoning. Refresh sessions. Show retries.	renderer
10	M-01, M-02	Steer long reasoning toward action. Discourage drafts in reasoning.	prompts, harness
11	E-02, E-03	Pair every result with a start frame. Add “wait for port or exit”.	protocol, jobs
12	S-02	Make the deletion-guard message correct.	permissions
13	E-05, E-04, E-06	Show the closest match in the Edit error. Track shell edits. Save statistics often.	tools, state
14	L-03, L-04, U-09	Smaller logs, durations, clear time zone, small UI defects.	logging, renderer

A Tool call counts

Tool	Calls	Tool	Calls
Write	56	Read	8
Edit	49	TaskStop	4
Bash	45	Grep	3
TaskUpdate	12	AskUserQuestion	2
TaskCreate	7	Glob, TaskOutput	1 each
Total	188	Errors	8

B Process measurements

Time	Renderer CPU	Renderer memory	Engine CPU	Engine memory
18:42	149 %	637 MB	2 %	123 MB
19:00:37	135 %	946 MB	1 %	130 MB
19:01:54	124 %	924 MB	—	—

A CPU value of 100 % is one full core.

C Glossary

Engine	The MAGENTRA process that runs the agent loop and the tools.
Renderer	The Electron process that draws the MAGENTRA window.
Main process	The Electron process that starts the engine and the window.
Turn	One user message and all agent work that follows it.
Delta frame	A small engine message with a part of the model output.
Backlog	Frames that arrived at the renderer but are not drawn yet.
Overdrive	A MAGENTRA mode. The agent runs commands without prompts and checks its result at the end.
Deletion guard	A MAGENTRA control. It asks the user before deletions.
Stale-read guard	A control that refuses an Edit when the file changed after the last Read.
Transcript	The JSONL record of a session.
Session log	The NDJSON record of the frames between the engine and the UI.

D Evidence files

- Session log: `test/.magenta/logs/desktop-20260923-180141.log`
- Transcript: `test/.magenta/sessions/s_mue8gg2u_5aed60.jsonl`
- Task file: `test/.magenta/tasks/s_mue8gg2u_5aed60.json`
- Game server log: `test/ServerLog.txt`
- Screenshots: `figures/` next to this report.